SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Manoj M	**Reg. No.:**	192511060
Section / Batch:	CSE	**Date:**	29.07.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q50. Debugging Insertion Sort Code (Wrong Direction in Shift Comparison)

The following Insertion Sort implementation uses an incorrect comparison condition in the while loop, causing smaller elements to remain in the wrong position and breaking ascending order. Carefully analyze the shifting logic and explain why the sorted prefix is not maintained correctly. Debug the algorithm step-by-step so that larger elements shift right and the key is inserted in the proper place. Optimize the implementation for clarity and maintain correct indexing throughout. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

Code of Conduct

I _Manoj M(192511060)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				

Total Marks (100):	

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Execution

Step 1: Identify the Error

The error is in the **while** condition.

while j >= 0 and arr[j] < key: # ERROR

For **ascending order**, larger elements should be shifted to the right. The correct condition is:

while j >= 0 and arr[j] > key:

Step 2: Why is it Wrong?

Example Array:

[5, 2, 4, 6, 1, 3]

Pass 1 (i = 1)

- Key = **2**
- j = 0
- Check: 5 < 2 → **False**

No shifting occurs.

Array remains:

[5, 2, 4, 6, 1, 3]

But **2** should be placed before **5**, so the sorted prefix is broken.

Step 3: Debugging

Problem

- Wrong comparison operator (<).
- Smaller elements are not inserted into their correct position.
- The sorted prefix is not maintained.

Fix

- Change the condition to arr[j] > key.
- Shift all larger elements one position to the right.
- Insert the key into its correct location.

Step 4: Optimized Code

```
def insertion_sort(arr):

    # Traverse from the second element
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Shift larger elements to the right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert key at the correct position
        arr[j + 1] = key

    return arr
```

Step 5: Time Complexity

- **Best Case: O(n)** (Already sorted)
- **Average Case: $O(n^2)$**
- **Worst Case: $O(n^2)$** (Reverse sorted)
- **Space Complexity: O(1)**

Final Result

The corrected Insertion Sort changes the comparison from arr[j] < key to arr[j] > key, ensuring that all larger elements are shifted right before inserting the key. This maintains the sorted prefix correctly, produces ascending order, and runs in **O(n)** in the best case and **$O(n^2)$** in the average and worst cases while using **O(1)** extra space.